# Homework 8

## 1 1. Maximum Clearance Routes

### 1.1 (a) Cut and Cycle Property of *maximum* spanning tree.

Maximum Spanning tree G: a tree that connects all vertices in G while maximizing the sum of the edge weights i

1. Cut Property: Let $i$ and $j$ be the nodes in the two sides of a cut. Let $e$ be the maximum weight across the cut. Then $e$ is in the Maximum Spanning Tree $G^*$
2. Cycle Property: Let $C$ be a cycle and let $f$ be the minimum weight edge in $C$. Then, $f$ is *not* in the Maximum Spanning Tree.

### 1.2 (b) MaxST Proof

Let $a$ be another route between $(u, v) \in G$ that has a greater route weight than the original edge between $(u, v)$, $o \in G$ and that it creates a cycle. Then, according to the cycle property of MaxST, we have to choose $a$ route to be MaxST. Here, using $o$ route contradicts the cycle property. Since $o$ is in the MaxST, $o$ must have the maximum route weight; otherwise there would be contradiction in the cycle property. Therefore, any graph $G$ with unique edge clearance and disntinct route weights, the path given by using edges from the maximum spanning tree of $G$ gives the maximum clearance route of $G$.

### 1.3 (c) MaxST Polynomial runtime Algorithm

#### 1.3.1 Pseudocode

**Algorithm 1:** MaxST($G$)

```
/* G: undirected, weighted adj list. Kruskal's Algoritm with edges sorted by
   decreasing order                                                        */
return ReverseKruskal(G);
```

#### 1.3.2 Runtime Analysis

The above pseudocode runs Kruskal's Algorithm with sorted edges by decreasing order. Since the sorting edges reversely does not change the runtime, the total runtime of the above algorithm is same as Kruskal's Algorithm's runtime: $O(mlog(m))$.

#### 1.3.3 Proof of Correctness

For the algorithm to always output the maximum spanning tree,

1. the output tree must correctly have the cut and cycle properties of MaxST.
   - the output tree must be acyclic.
2. all nodes must be connected

3. the sum of the edge weights of the tree in the graph $G$ must be greater than or equal to the sum of edge weights of any other spanning tree in the graph $G$.

According to the proof of correctness of Kruskal's algorithm's first case that checks the existence of cycle, there exists no edge that creates cycle such that the output tree is always acyclic.

Moreover, for all nodes to be connected, there needs to be $n-1$ edges when there exists $n$ nodes in the tree from the graph $G$. Kruskal's Algorithm ensures to stop searching when $|T| = n - 1$ where $T$ is the output tree as we are always given the graph $G$ with distinct weights for each edge in the graph which ensures no overlaps in edge weights. Reversing the node alignment for searching does not affect the stop condition of the algorithm. Therefore, all nodes are always connected in the output tree.

Lastly, the algorithm greedly selects nodes that does not create cycles from sorted edges in decreasing order. According to the cycle and cut properties of MaxST, there cannot be edges with minimum weights in MaxST and there must be edges with maximum weights in MaxST. Therefore, the sum of the edge wegiths in the output tree must be greater than or equal to any other possible spanning tree from the graph $G$.

Therefore, `MaxST(G)` always outputs the correct maximum spanning tree from any arbitrary graph $G$ that has dintinct edge weights in each edge of it.

# 2 Streaming Edges

## 2.1 (a)

Problem: Given $G, T, (u, v), w(e)$, how is it possible to get a MST $T'$ for the larger graph $G'$ in time $O(n)$ where $n$ is the number of nodes in $G$.

### 2.1.1 Facts

1. $G = (V, E)$: undirected graph with edge weights $w$
2. Already computed MinST, $T$ for $G$.
3. $G' = G$ with a new edge $e = (u, v)$ with weight $w(e)$
4. $T'$ is given as a graph with the same node set as $G$
5. $O(n)$ is not enough to re-read the original graph $G$. Have to explain why OK to ignore lots of the edges in $G$.

### 2.1.2 Proof

Getting $G'$ by adding additional edge $e = (u, v)$ with weight $w(e)$ from the original graph $G$ between any existing nodes in $G$, $u, v$ creates a cycle since MST from any graph is unique.

To check whether we should keep the new edge $(u, v)$ and which edge to delete iff the new edge $(u, v)$ has the minimum weight wihtin the cycle $C$, check all nodes that has edges and that create the cycle $C$ from either $u$ or $v$ (it does not matter where to check from).

- check all nodes from either $u$ or $v$ and see if the edge weight between either $u$ or $v$ and any nodes in $C$ is greater than $(u, v)$.

- if greater than $(u, v)$: delete the heaviest edge and keep $(u, v)$ from $T$ to create new $T'$
- else: the original $T$ is $T'$.

Because the worst case of this solution checks all nodes in the graph, it runs $O(n)$ in worst case.

## 2.2 (b)

Write a MST algorithm with lazy operations with streams.

1. $edge_stream$: stream data structure

- $edge_stream.has_next()$:
  - return type: `boolean`
  - checks if there are more edges in the stream
- $edge_stream.next_edge()$:
  - return type: `pair`
  - returns the next edge in the stream

2. cannot go backwards and re-read previous edges

- store them if we want to re-access

### 2.2.1 Pseudocode

**Algorithm 2:** LazyMST($V$, *edge_stream*)

```
   /* overall idea:                                                          */
   /* 1.  compare weights of its available streamed edges                    */
   /* 2.  pick the minimum edge between u and vand exclude the edges that cannot be in
      MST or the ones we already checked                                     */
   /* 3.  add it to MST                                                      */
 1 MST ← empty adjacency list;
   /* first, we need to initialize MST adj list with nodes:  space:  O(n), time:
      O(n)                                                                   */
 2 foreach each vertex n in V do
 3     MST[n] ← empty list;
   /* checking for each edge:  O(m)                                          */
 4 while edge_stream.has_next() do
 5     (u, v, w) ← edge_stream.next_edge();
 6     path ← partialDFS(MST, u, v);
       /* the path u ⇝ v found by DFS. Additional edge weight tracking added */
 7     if path = null then
 8         MST[u] ← (v, w);
 9         MST[v] ← (u, w);
10     else
11         edge ← path[0];
12         max_weight ← edge[2];
13         foreach parent, child, weight in path do
14             if weight > max_weight then
15                 max_weight ← weight;
16                 edge ← (parent, child, weight);
           /* this ensures that we do not check for edges that we already checked or */
           /* that cannot be in MST so that we do not have to check it ever in the
              stream                                                         */
17         if w < max_weight then
18             remove edge from both endpoint lists in MST;
19             MST[u] ← (v, w);
20             MST[v] ← (u, w);
21 return MST;
```

**Algorithm 3:** partialDFS($MST$, $u$, $v$)

```
   /* Run DFS starting at u in the current forest.  Each MST[u] entry is          */
   /* (connecting nodes, weight).  Return the path as a list of connected edges.  */
   /* variation of original DFS; it finds the path from u to v only.              */
 1 parent_edge ← hash table;
   /* we need hash table that keeps track of edges between nodes on top of family
      relationship                                                                 */
 2 found ← DFS-visit(u, v);
   /* boolean value to see if there exists a path                                 */
 3 if found then
       /* if we have path, we need the subgraph S to find the optimal path in LazyMST
          */
 4     path ← empty list;
       /* insert all the connecting nodes between u and v                         */
 5     con_nodes ← v;
 6     while con_nodes ≠ u do
 7         path.append(parent_edge[con_nodes]);
 8         con_nodes ← parent_edge[con_nodes][0];
           /* move to the parent endpoint                                         */
 9     $;
10     return path;
   /* if not found, then null                                                     */
11 return null;
```

**Algorithm 4:** DFS-visit($u$, $v$)

```
   /* base case to stop recursion                                                 */
 1 if u = v then
 2     return true;
   /* recursively search all nodes that from u to v                               */
   /* we may have connecting nodes between u and v                                */
 3 foreach (con_node, w) in MST[u] do
       /* If the connecting node has not been visited, record its parent edge.    */
 4     if con_node ∉ disc then
 5         parent_edge[con_node] ← (u, con_node, w);
 6         if DFS-visit(con_node, v) then
 7             return true;
 8 return false;
```

### 2.2.2 Runtime/Space Analysis

**Space**

1. $O(n)$: we use adjacency list `MST` to store $n$ nodes
2. $O(n)$: `parent_edge` takes $n$ number of edge entries at most in stream. For each node search for minimum weight, it gets initialized such that it never exceeds $O(n)$. Therefore the space complexity of this algorithm is $O(n)$.

**Runtime**

1. $O(n)$: MST initialization with $n$ entries
2. $O(m)$: outer loop iterating through $m$ edges in the stream
3. $O(n)$: `partialDFS` runs $O(n)$ times for each iteration of outer loop
4. $O(m)$: parallel to the `partialDFS` recursive call; in worst case, we iterate through the entire edge.

$O(n) + O(m) \cdot (O(n) + O(m)$(where n ¿ m)) $= O(nm)$ Therefore, the total runtime of this algorithm is $O(nm)$.

### 2.2.3 proof of correctness

For this algorithm to always output the correct MST, the MST for each processed streamed edge must be a minimum spanning forest of the graph given formed by i edges where i is the number of processed, streamed edges so far.

**Proof by induction**

**Base case** if the forest is empty, then it is a minimum spanning forest of the empty graph

**Inductive Step**

- for $k$th edge $(u_k, v_k, w_k)$, if $u_k$ and $v_k$ are not connected yet in MST, connect.
- Otherwise, there will be cycle.
- for any other edges that is on the path from $u$ to $v$, if adding it to MST creates a cycle, by the cycle property of MST, the edge with maximum weight within the comparison cannot be in MST.
- So, we exclude those edges from the stream such that the minimum edge weight gets preserved in each edge of the forest.
- So, after the terminating the stream as we went through all possible edges in the graph, there will only be edges with minimum edge weights left.

Therefore, the algorithm always correctly computes MST.